

MODULE :- Automation with Selenium

EXPERIMENT TITLE :- Synchronization & Explicit Waits (Assignment 2)

1.Problem Statement :-

To automate a web page containing dynamically loaded content using **Selenium WebDriver** and **explicit waits**. The test should wait for the required web element to become available or visible before interacting with it, without using `time.sleep()`, and verify that the dynamic content is loaded successfully.

2. Tools and Software Required :-

S. No.	Tool / Software	Purpose
1	Python	Programming language used to write the automation script
2	VS Code	IDE used to write and execute Python code
3	Google Chrome	Web browser used for automation
4	Selenium WebDriver	Automates the browser and interacts with web elements
5	WebDriverWait	Waits for a specified condition before continuing execution
6	Expected Conditions	Used to wait for conditions such as element visibility and clickability
7	Command Prompt / Terminal	Used to run the Python automation script
8	Test Automation Practice Website	Website used as the application under test

3. Concepts Used :-

- **Selenium WebDriver** – Used to automate and interact with the web browser.
- **Dynamic Elements** – Web elements whose availability or content may change after the page loads.
- **Explicit Wait** – Used to wait until a specific condition is satisfied before proceeding.
- **WebDriverWait** – Defines the maximum time Selenium should wait for a specified condition.
- **Expected Conditions** – Used to check conditions such as element visibility and clickability.
- **Synchronization** – Ensures that Selenium interacts with dynamic content at the appropriate time.

4.Source Code :-

File 1: assignment_2.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.maximize_window()

try:
    wait = WebDriverWait(driver, 15)

    driver.get(
        "https://testautomationpractice.blogspot.com/"
        "p/gui-elements-ajax-hidden.html"
    )

    # Wait for AJAX button
    ajax_button = wait.until(
        EC.element_to_be_clickable(
            (
                By.XPATH,
                "//button[contains(., 'Load AJAX Content')]"
            )
        )
    )

    ajax_button.click()

    print("AJAX button clicked")

    # Wait for status
    status = wait.until(
        EC.visibility_of_element_located(
            (
                By.XPATH,
                "//*[contains(text(), 'Status:')]"
            )
        )
    )
```

)

```
print("Status:", status.text)
```

```
print("PASS: Explicit wait completed successfully")
```

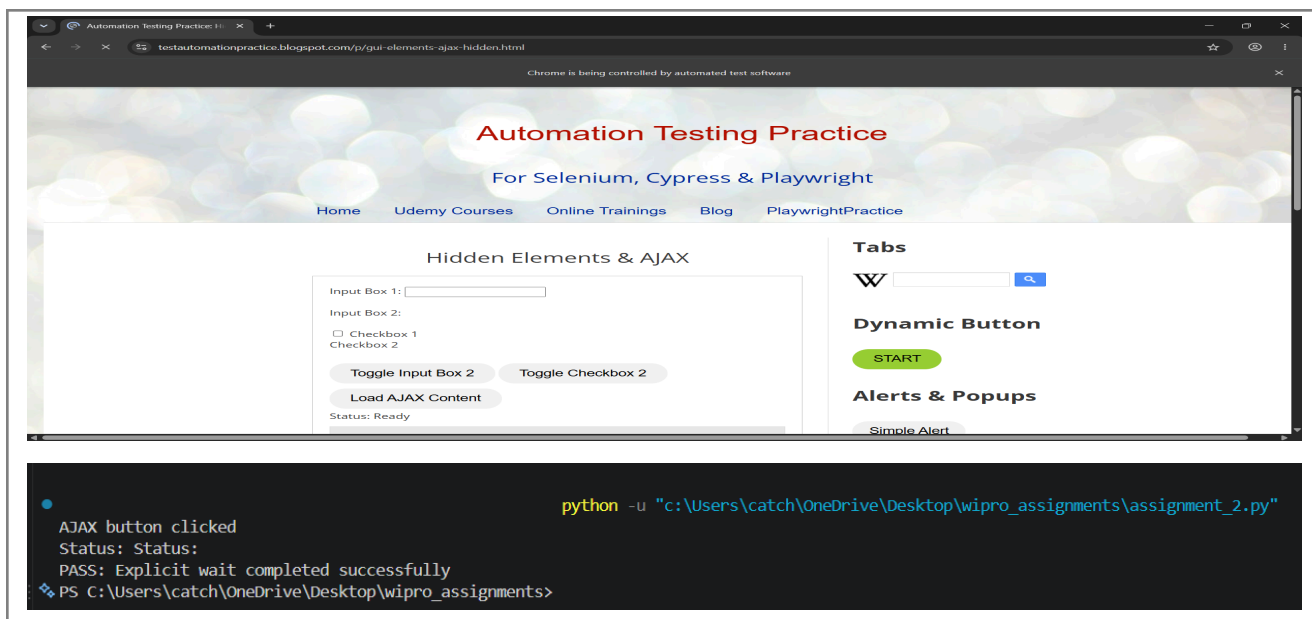
finally:

```
driver.quit()
```

5.Implementation Steps :-

- 1.Install Python and Selenium WebDriver.
- 2.Open the Test Automation Practice page containing dynamically loaded content.
- 3.Create a WebDriverWait object with the required maximum wait time.
- 4.Locate the required dynamic element using an appropriate Selenium locator.
- 5.Use an Expected Condition to wait until the element is ready for interaction.
- 6.Interact with the element after the required condition is satisfied.
- 7.Wait for the dynamically generated content to become visible.
- 8.Retrieve and display the loaded content.
- 9.Verify the result using an assertion or successful element retrieval.
- 10.Close the browser after completing the test.

6.Output :-



7.Result :-

The Selenium automation test was successfully executed to handle dynamically loaded web content using **explicit waits and expected conditions**.

8.Observation :-

The test waited for the required condition instead of relying on fixed delays. This allowed Selenium to interact with the dynamic element when it became available.

9.Conclusion :-

Thus, **Selenium WebDriver with WebDriverWait and Expected Conditions** was successfully used to synchronize automation with dynamically loaded web content.